

Secure Software Design and Engineering (CY-321)

Keep it Simple, Stupid

Subjects and Objects

Most access control is concerned with (active) *subjects* getting access to (passive) *objects*.

A web server, acting on behalf of user, changes a row in a database.

An ATM, acting on behalf of you, reduces your account balance by some amount. . .

Subjects and Objects

Most access control is concerned with (active) *subjects* getting access to (passive) *objects*.

A web server, acting on behalf of user zubair, changes a row in a database.

An ATM, acting on behalf of you, reduces your account balance by some amount. . . and hopefully hands out some cash in the process

Mechanism and Policy

Mechanism tells us *how* access control is enforced.

Policy tells us which subjects get access to which objects.

Separating How from What/Who

If it is (or should be) in code, it's a **mechanism**. If it is (or should be) in a config file, it's a **policy**.

Mechanism and Policy

It is important to separate mechanism and policy:

- Debug both separately
- Protect one from bugs in the other
- Change the policy without having to change the mechanism

It's important to separate the two *completely*

Access Control Matrix

View of access control mechanisms as matrix: row contains **active objects** (users, processes, etc.), columns contain **passive objects** (files etc.), matrix entry says what active object may do with passive object.

	passwd	httpd.conf	ls	Objects
root	r/w	r/w	r/w/x	
zubair	r	r	r/x	
Waqar	r	r/w	r/x	
Amjad	r	r/w	r/x	
Subjects				

Full Access Control Matrix

A full access control matrix with n subjects and m objects has nm entries.

Typical Linux system has 100,000 files and directories in the root filesystem (not counting home directories, devices, mail spool etc.) and 30 entries in the password file: 3,000,000 entries, most of which are empty.

Every new user adds 100,000 entries.

Solution 1: Store entries by row: *capability*

Solution 2: Store entries by column: *access control list*

Solution 3: Store mostly columns, compute some rows: *hodgepodge :-)*

Access Control Matrix

View of access control mechanisms as matrix: row contains **active objects** (users, processes, etc.), columns contain **passive objects** (files etc.), matrix entry says what active object may do with passive object.

	passwd	httpd.conf	ls	Objects
root	r/w	r/w	r/w/x	
zubair	r	r	r/x	Capability
Waqar	r	r/w	r/x	
Amjad	r	r/w	r/x	
Subjects		ACL		

Solution 1

Solution 2

Properties of Capability

Capability systems issue a capability to a subject. When the subject wants to access an object, it presents the capability. This capability could be encrypted or otherwise protected.

- Can be given to other subjects (which can be **good** or **bad**)
- Can sometimes be computed on the fly (need not be stored)
- Can be issued minimally, i.e., just enough capabilities to do the job, but no more
- Revocation of capabilities and audit (who can do what) **tricky**
- Dissemination not easily controllable.

Crude example: **passwords** (no fine-grained control)

Properties of Capability

	File A	File B	File C
Alice	Read, Write	Read	—
Bob	Read	Read, Write	Execute
Carol	—	—	Read, Write, Execute

Alice's Capabilities:

File A → Read, Write

File B → Read

By row (per subject)

Properties of ACL

	File A	File B	File C
Alice	Read, Write	Read	—
Bob	Read	Read, Write	Execute
Carol	—	—	Read, Write, Execute

The ACL for **File A**:

File A:

Alice → Read, Write

Bob → Read

By column (per object)

Properties of ACL

ACLs are dual to capabilities:

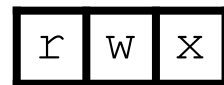
- ACLs cannot be transferred to other subjects (which can be **good** or **bad**)
- Must usually be stored with the object
- Creating minimal ACLs is difficult
- Revocation of access rights and audit easy
- Dissemination impossible, trivially controllable
- Privilege-granting privilege cannot be constrained

Crude example: Unix permission bits

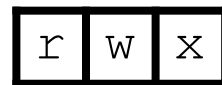
Unix file objects have an owner and a group. They also have nine bits associated with them



Owner



Group



Others

r = read

w = write

x = execute

Meaning of Bits

The first group of three bits tell what the *owner* of the object may do with it.

The second group of three bits tell what *group* members may do with it.

The third group of three bits tell what all *others* may do with it.

Within a group, bit 0 means “execute permission” (on directories: can change to this directory)

Bit 1 means “write permission” (on directories: can create new files, delete files)

Bit 2 means “read permission” (on directories: can read directory contents)

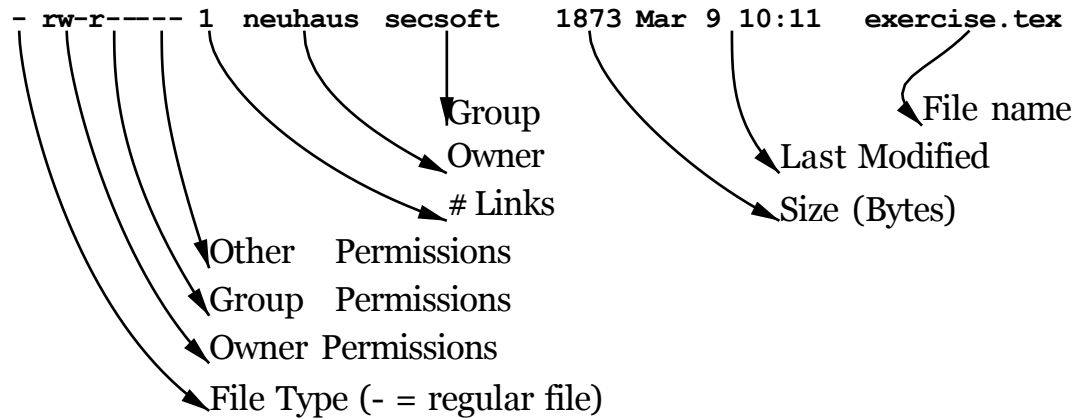
Access Control

When a file is opened (removed, renamed, executed, attributes changed), the process requesting the operation makes a *system call*.

The operating system examines the *effective user ID* (euid) and *effective group ID* (egid) of the process making the call.

It also looks at the file's (or directory's) *attributes* and thereby decides whether to perform the operation or not.

Example



PID 123 has euid zeller and egid secsoft ϵ access granted

Process 234 has euid cleve and egid users ϵ access denied

Process 345 has euid neuhaus ϵ access granted

Process 1 has euid root ϵ access granted

Numerical Modes

Since there are groups of three bits each, we can express a mode in base 8:

Mode	Binary	Octal
<code>rw-r--r--</code>	<code>110100100</code>	<code>644</code>
<code>rw-----</code>	<code>110000000</code>	<code>600</code>
<code>rw-r-xr-x</code>	<code>111101101</code>	<code>755</code>
<code>rw-rwx---</code>	<code>111111000</code>	<code>770</code>
<code>--x--x--x</code>	<code>001001001</code>	<code>111</code>

Additional Bits

Bit No. 10: Sticky bit. On regular files mostly without semantics; on directories: only owner can delete file, regardless of mode (used for temp directories)

Letter: t (if other x bit set) or T (if other x bit not set)

/tmp has mode 1777, /tmp/strange has mode 1770.

```
drwxrwxrwt  9 root      root    16384 Mar  9 13:01 /tmp
drwxrwx--T  2 neuhaus  users    4096 Mar  9 13:39 /tmp/strange
```

setuid Explained

Unix is a multi-user OS. Different users have different numerical user IDs (called UIDs) and can be members of various *groups*, which also have different numerical group IDs (called GIDs).

One user has ultimate power over a machine. This is the *superuser* (aka *root*) which has UID 0.

When a new process is created, it inherits its UID and GID from the process that created it (the *parent process*).

The initial process (surprisingly called `init`) has UID 0.

In order to separate different users, a process running with UID 0 can set its UID with a system call (called `setuid()`). Once a program has given up its privileges, it can usually not revert to its old ones.

setuid Explained

When a program is executed, it is loaded (e.g., from disk), turned into a process, and started.

Some processes need more privileges than the parent process has.

For example, a process that changes a user's password needs superuser privileges because only the superuser can write the password file.

Some programs are therefore flagged with the additional information that any processes created from them need to have their UID changed to the owner of the program, and not inherited from the parent process.

This privilege elevation (and sometimes demotion) is handled by the kernel.

Mapping Unix to Access Matrix

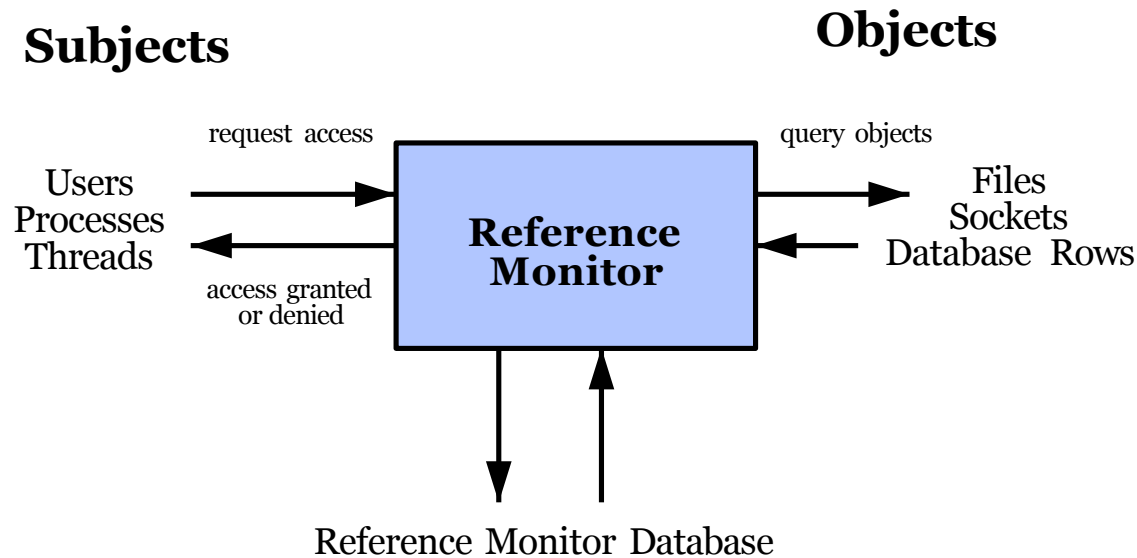
The Unix permissions are (mostly) associated with objects. It is therefore (mostly) an ACL-like system.

The Access Control List then contains the owner explicitly, and the members of the file's group and everybody else implicitly.

Windows NT (and, by extension, XP and 2003) have real ACLs, where every object has a list of subjects and its permissions.

Reference Monitors

The reference monitor is that piece of software that performs the access decision.



It implements the mechanism.

- Mediate *every* access
- Tamper-proof
- Simple enough to be analyzed comprehensively

REFERENCES

- Peter Gutmann, *Cryptographic Security Architecture*, Springer
- Peter Gutmann, *cryptlib Encryption Toolkit*,
<http://www.cs.auckland.ac.nz/~pgut001/cryptlib/>

Questions??

zubair.ahmad@giki.edu.pk

Office: G14 FCSE lobby